

גיא נאור- סיכום שיעור 8 - flexible schemes

בשיעור זה תחילה דיברנו על פרויקט הסיום, לאחר מכן שאלנו שאלות לגבי המטלה של הדירוג זוגות. אחר כך עברנו ללמוד סכמות גמישות (json) וקבועות (sql), תחילה למדנו בכלליות על יתרונות וחסרונות ואז עברנו לדוגמאות וכתובת קוד שממחיש את השוני בין הסכמות. לבסוף עברנו לתרגל מבחן לדוגמא, את החלק של sql מועד א 2025.

סכמות גמישות וקבועות

אחד היתרונות המרכזיים של בסיסי נתונים רלציוניים (SQL) הוא שהסכמה שלהם קבועה מראש. כלומר, לכל הרשומות בטבלה יש בדיוק את אותן העמודות ואותם סוגי הנתונים.

עם זאת, יש מקרים שבהם סכמה קבועה מגבילה אותנו, ולכן נעדיף להשתמש בסכמה גמישה. בסכמה גמישה אין מבנה קבוע מראש, וכל רשומה יכולה להכיל שדות שונים בהתאם לצורך.

כאשר משתמשים בסכמה גמישה אין לנו את "חגורת הבטיחות" של בסיס הנתונים. המערכת לא תמיד יודעת לוודא שהנתונים תקינים, ולכן האחריות עוברת יותר לקוד של המערכת. אם לא נבצע בדיקות מתאימות, עלולות להופיע שגיאות ובעיות באיכות הנתונים שיהיה קשה לאתר בהמשך, במיוחד כאשר עובדים עם כמויות גדולות של מידע.

ניתן לחשוב על ההבדל כך:

- **סכמה קבועה (SQL)** – כמו טופס שבו כל המשתמשים חייבים למלא את אותם השדות.
- **סכמה גמישה** – כמו מסמך חופשי שבו כל משתמש יכול להוסיף מידע שונה.

מתי נרצה להשתמש בסכמה גמישה?

- כאשר מבנה הנתונים עדיין לא ידוע במלואו.
- כאשר המוצר נמצא בשלבי פיתוח והדרישות משתנות במהירות.
- כאשר יש צורך לתמוך במספר סוגי רשומות בעלי מבנים שונים.
- כאשר הנתונים הם היררכיים או מקוננים (Nested Data).

לדוגמה, בתחילת פיתוח מוצר ייתכן שקיימים רק השדות A, B ו-C, אך חודש לאחר מכן נוסיף גם D, E ו-F. בסכמה גמישה אין צורך לשנות את מבנה הטבלה בכל פעם שמתווסף שדה חדש.

למשל: אם מחר נרצה להוסיף שדה חדש כמו Birthday, נצטרך לשנות את מבנה הטבלה.

Id	Name	Age
1	Guy	27

דוגמה נוספת היא יחידות צבאיות:

— גדוד L

— פלוגה L

— מחלקה L

— כיתה L

מידע כזה הוא היררכי ומקונן, ולכן לעיתים נוח יותר לייצג אותו באמצעות סכמה גמישה.

JSON ו-XML

הדרך הנפוצה ביותר לייצג סכמות גמישות כיום היא באמצעות JSON.

JSON פותח בשנת 2001, והוא אחד הפורמטים הנפוצים ביותר להעברת ואחסון מידע במערכות מודרניות. למרות שהוא טקסט רגיל, הוא **מובנה** כך לייצג מבנים מורכבים של אובייקטים, רשימות ונתונים מקוננים.

MySQL תומך באופן מובנה ב-JJSON ואף מספק פונקציות לשליפה וחיפוש של מידע מתוך מסמכי JSON.

לפני JSON היה נפוץ מאוד פורמט בשם XML. שני הפורמטים דומים ברעיון ומאפשרים לייצג נתונים היררכיים, אך כיום JSON נחשב פשוט יותר ולכן נפוץ הרבה יותר בפיתוח מערכות מודרניות.

דוגמא לjson :

Json example

```
{ "first_name": "John",  
  "last_name": "Smith",  
  "is_alive": true,  
  "age": 27,  
  "address": {  
    "street_address": "21 2nd Street", "city": "New York",  
    "state": "NY", "postal_code": "10021-3100" },  
  "children": ["Catherine", "Thomas", "Trevor"],  
  "spouse": null  
}
```

סכמות גמישות (Flexible Schemas)

יתרונות

1. מתאימה כאשר המבנה לא ידוע מראש או משתנה לעיתים קרובות

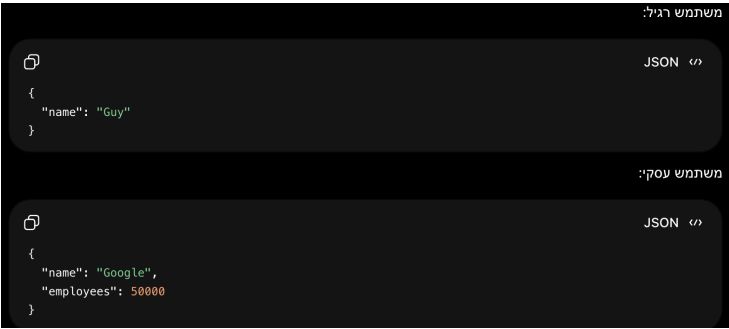
אם בונים אפליקציה חדשה ועדיין לא יודעים אילו נתונים יישמרו בעתיד, קל יותר להשתמש בסכמה גמישה.

דוגמה:

רשת חברתית חדשה שבה כל כמה שבועות מוסיפים מאפיינים חדשים לפרופיל המשתמש.

2. ניתן לשמור כמה סוגי מבנים באותה טבלה

לא כל הרשומות חייבות להכיל את אותם השדות.



שניהם יכולים להישמר יחד ללא שינוי של מבנה הנתונים.

3. חוסכת מקום כאשר יש הרבה שדות ריקים

ב-SQL לעיתים יש הרבה עמודות שרובן ריקות.

טבלת רכבים:			
Battery	Engine	Color	Id
NULL	V8	Red	1
80	NULL	Blue	2

בסכמה גמישה נשמור רק את המידע שקיים בפועל. (ללא הnull).

4. פחות צורך ב-JOIN

אפשר לשמור מידע קשור יחד באותו מסמך.

```
JSON <>
{
  "name": "Guy",
  "address": {
    "city": "Tel Aviv",
    "street": "Herzl"
  }
}
```

במקום טבלת משתמשים וטבלת כתובות, אין צורך לבצע JOIN כדי לקבל את הכתובת.

חסרונות

1. מאבדים יתרונות של מסדי נתונים רלציוניים

המערכת פחות שומרת על תקינות הנתונים. שמות השדות הם case sensitive



אם נרצה לחשב ממוצע גילאים זאת תהיה בעיה, כי שמות השדות לא אותו דבר.

2. צריך לכתוב יותר קוד באפליקציה

במקום שהמסד יבדוק שהנתונים תקינים, המתכנת צריך לבצע את הבדיקות בעצמו.

3. פחות נוח לניתוחים ושאלות

ב-SQL קל לבצע חישובים, סינונים וקיבוצים.

כאשר לכל רשומה יש מבנה שונה, פעולות כאלה הופכות למורכבות יותר.



4. פחות תמיכה במנגנונים מתקדמים

מסדי נתונים רלציוניים מספקים מנגנונים חזקים של אבטחה, הרשאות, נעילות משתמשים, ביצועים ועסקאות (Transactions).

בסכמות גמישות לעיתים חלק מהיכולות הללו מוגבלות יותר.

מתי נעדיף כל אחד?

- **SQL / Relational Database** – כאשר המבנה ידוע מראש וחשובים לנו סדר, אמינות וניתוח נתונים.
- **Flexible Schema** – כאשר המבנה משתנה לעיתים קרובות או כשלא יודעים מראש אילו שדות יהיו בעתיד.

שימוש בשני העולמות יחד

בפועל, במערכות מודרניות רבות לא בוחרים בין בסיס נתונים רלציוני לבין סכמה גמישה, אלא משלבים ביניהם.

המידע החשוב והקבוע נשמר בעמודות SQL רגילות:

Id
Name
Email

לעומת זאת, מידע שמשתנה בין משתמשים או אינו קיים אצל כולם נשמר בשדה JSON.

לדוגמה, משתמש אחד יכול להכיל:

```
JSON </>
{
  "skills": ["SQL", "Python"],
  "city": "Tel Aviv"
}
```

ומשתמש אחר:

```
JSON </>
{
  "hobbies": ["Gym", "Music"],
  "favorite_color": "Blue"
}
```

גישה זו מאפשרת לקבל את היתרונות של שני העולמות:

מצד SQL:

- תקינות נתונים
- ביצועים טובים
- JOIN
- GROUP BY
- אבטחה והרשאות

מצד JSON:

- גמישות
- הוספת שדות חדשים ללא שינוי הטבלה
- שמירת מידע היררכי ומקונן (Nested Data)

בשיעור הודגם כי MySQL תומך באופן מובנה ב-JSON. כאשר שומרים JSON בעמודה מסוג JSON, MySQL בודק שהמידע תקין ומאפשר לשלוף ממנו שדות ספציפיים באמצעות פונקציות ייעודיות.

לכן כיום מקובל להשתמש בבסיס נתונים רלציוני כבסיס המערכת, ובשדות JSON עבור מידע גמיש שאינו מתאים לסכמה קבועה.

כתיבת קוד

בחלק הזה של השיעור עברנו לכתוב קוד גם בפורמט json וגם בsql ועשינו השוואה והדגמנו מקרים ותקלות כדי להבין יותר לעומק את ההבדלים.

1. שמירת JSON כטקסט רגיל

```
SQL </>  
  
CREATE TABLE flex (  
    meaning varchar(255),  
    flex_schema varchar(1000)  
);
```

עד עכשיו הכרנו טבלאות SQL שבהן כל עמודה מכילה סוג נתונים מסוים, למשל מספר או טקסט. כאן יצרנו עמודה בשם flex_schema מסוג varchar, כלומר טקסט רגיל. המטרה היא להראות שאפשר לשמור JSON גם בתור טקסט רגיל, בדיוק כמו שהיינו שומרים משפט או פסקה.

2. הכנסת JSON לא תקין

```
SQL </>  
  
insert into flex values (  
    'invalid json',  
    '{"first_name": "John",  
    "last_na  
' );
```

למרות שה-JSON שבור ולא תקין, MySQL עדיין מכניס אותו לטבלה.

הסיבה היא שהעמודה מוגדרת כטקסט רגיל (varchar), ולכן MySQL לא בודק את תקינות ה-JSON.

זוהי אחת הסכנות של סכמות גמישות כאשר לא משתמשים במנגנוני בדיקה.

3. יצירת טבלה עם עמודת JSON אמיתית

במקום לשמור JSON כטקסט, אפשר להגדיר שהעמודה עצמה היא מסוג JSON.

במצב כזה MySQL מבין שמדובר ב-JSON ומתחיל לבדוק את תקינות המבנה.

```
SQL <>
CREATE TABLE flex (
  meaning varchar(255),
  flex_schema JSON
);
```

4. ניסיון להכניס JSON לא תקין

הפעם הפקודה תיכשל בניגוד לדוגמא קודמת.

בניגוד לעמודת טקסט רגילה, עמודת JSON לא מאפשרת לשמור מידע שאינו JSON תקין.

כלומר, קיבלנו חלק מההגנה והבקרה שמסד נתונים רלציוני יודע לספק.

```
SQL <>
insert into flex values (
  'invalid json',
  '{"first_name": "John",
  "last_na
');
```

5. מספר ערכים לאותו שדה

גם כאשר ה-JSON תקין מבחינה תחבירית, עדיין יכולות להיות בעיות באיכות הנתונים.

בדוגמה הזאת אותו שדה מופיע מספר פעמים.

הדבר מדגים שסכמה גמישה נותנת הרבה חופש, אך גם דורשת יותר אחריות מצד המתכנתים.

```
SQL <>
insert into flex values (
  'multiple values for the same key',
  '{
    "first_name": "John",
    "first_name": "Not John",
    "first_name": "Maybe John"
  }'
);
```

6. הצגת כל הרשומות בטבלה

למרות שהמידע נשמר כ-JSON, עדיין מדובר בטבלת MySQL רגילה.

אפשר לבצע עליה שאילתות SQL בדיוק כמו על כל טבלה אחרת.

```
SQL </>

select *
from flex;
```

7. גישה לשדות בתוך JSON

אפשר לגשת ישירות לשדות שנמצאים בתוך ה-JSON.

בדוגמה זו:

- השורה הראשונה מחזירה את כל הכתובת.
- השורה השנייה מחזירה רק את העיר.

הדבר דומה לגישה לשדות בתוך אובייקט בתכנות.

```
SQL </>

select
flex_schema->"$.address" as address,
flex_schema->"$.address.city" as city
from flex
where meaning = 'Person';
```

8. גישה לשדה שלא קיים

בסכמה גמישה לא כל שדה חייב להופיע בכל רשומה.

כאשר מבקשים שדה שלא קיים, MySQL מחזיר NULL ולא שגיאה.

כך ניתן לעבוד גם כאשר למשתמשים שונים יש מבנים שונים.

```
SQL </>

select
flex_schema->"$.non_existing_field"
from flex
where meaning = 'Person';
```

9. בדיקה האם שדה קיים

לפני שמשתמשים בשדה מסוים, אפשר לבדוק האם הוא בכלל קיים. זה שימושי במיוחד כאשר עובדים עם סכמות גמישות שבהן לא כל הרשומות נראות אותו הדבר.

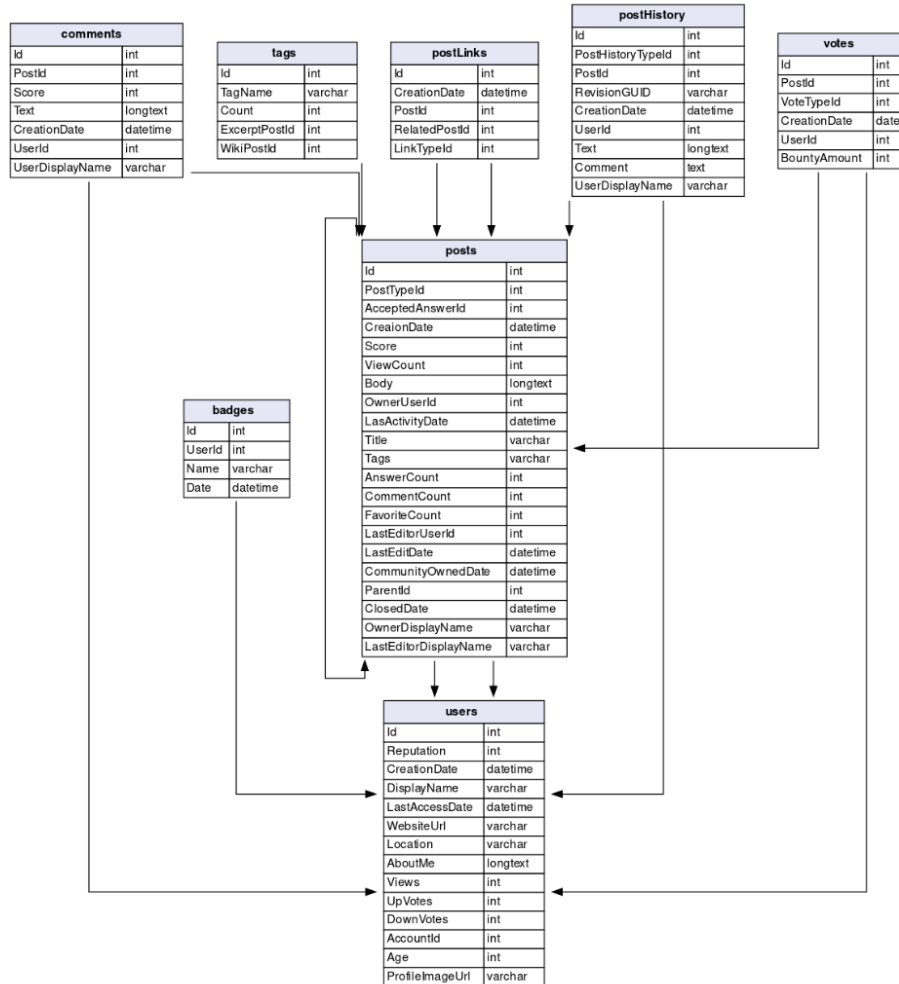
הפונקציה מחזירה:

1 = קיים
0 = לא קיים

```
select
flex_schema->"$.address" as address,
JSON_CONTAINS_PATH(
    flex_schema,
    'one',
    "$.address"
) as is_key_there
from flex;
```

מבחן דוגמא - מועד א 2025

תרגלנו [מבחן לדוגמא](#) כדי לחזור על החומר וכתבתי שאלות לקראת המבחן.



a. שילפו את מספר ה'tag' ששמו sql

```

select
id
from tags
where lower(TagName) = 'sql'
;
    
```

תחילה פנינו לטבלת tags מכיוון שהיא מכילה את כל התגים במערכת. לאחר מכן סיננו את הרשומות כך שיוחזר רק התג ששמו sql. השתמשנו בפונקציה lower כדי להתעלם מהבדלים בין אותיות גדולות וקטנות. לבסוף שלפנו את מזהה התג.

b. שילפו את מספר הusers

```
select count(distinct Id) as users_count  
from users ;
```

פנינו לטבלת users מכיוון שהיא מכילה את כל המשתמשים במערכת. לאחר מכן השתמשנו בפונקציית count כדי לספור את מספר המשתמשים. השימוש ב-distinct מבטיח שכל משתמש ייספר פעם אחת בלבד.

c. שילפו את הusers שהם לא owners של אף post

```
select u.Id  
from users as u  
left join posts as p  
on p.OwnerUserId = u.Id  
where p.OwnerUserId is null ;
```

התחלנו מטבלת users משום שרצינו לבדוק את כל המשתמשים במערכת. לאחר מכן ביצענו left join לטבלת posts על פי מזהה המשתמש. בעזרת החיבור הצלחנו לזהות משתמשים שאין להם פוסטים. לבסוף סיננו את התוצאות כך שיוחזרו רק המשתמשים שלא נמצאה להם התאמה בטבלת posts.

d. שילפו את מספר הcomments לכל post, בעזרת טבלת comments

```
Create view cc as  
select p.Id, count(c.Id) as comments_count  
from posts as p left join comments as c  
on p.Id = c.PostId  
group by p.Id ;
```

התחלנו מטבלת posts כדי לכלול את כל הפוסטים במערכת. לאחר מכן ביצענו left join לטבלת comments על פי מזהה הפוסט. ספרנו את מספר התגובות לכל פוסט באמצעות count וקיבצנו לפי מזהה הפוסט באמצעות group by. לבסוף שמרנו את התוצאה ב-view בשם cc.

e. עבור posts שקיבלו הערות, שילפו את הcomments per post distributions, כלומר מספר הפעמים שpost קיבל הערה 1,2,3 וכדומה

```
select comments_count, count(PostId) as posts_count
from(
    select PostId, count(*) as comments_count_
    from comments group by PostId) as cc
group by comments_count
order by comments_count ;
```

תחילה השתמשנו בטבלת comments כדי לחשב כמה תגובות קיבל כל פוסט. לאחר מכן קיבצנו את הפוסטים לפי מספר התגובות שקיבלו. לבסוף ספרנו כמה פוסטים קיימים בכל קבוצת תגובות והצגנו את ההתפלגות לפי מספר התגובות.

f. הרחיבו את הסעיף הקודם לposts שלא קיבלו הערות

– option 1 - without using the view in d

```
select comments_count, count(PostId) as posts_count
from
    ( select p.Id, count(c.Id) as comments_count
    from posts as p
    left join comments as c
    on p.Id = c.PostId
    group by p.Id) as cw0
group by comments_count
order by comments_count ;
```

– option 2 - it is better because it saves time- using the view from d

```
Select comments_count, count(*) as total_posts_count
From cc
Group by comments_count
Order by comments_count
;
```

כדי לכלול גם פוסטים שלא comments לטבלת left join וביצענו posts באפשרות הראשונה התחלנו מטבלת קיבלו תגובות. לאחר מכן חישבנו את מספר התגובות לכל פוסט ובנינו את ההתפלגות. באפשרות השנייה שנוצר קודם לכן, קיבצנו לפי מספר התגובות וספרנו כמה פוסטים קיימים בכל קבוצה. view השתמשנו ב פתרון זה יעיל יותר מכיוון שהתוצאה כבר חושבה מראש

g. הציגו את זוגות post idsn שיותר מ10 אנשים זהים הצביעו לשניהם (כלומר אותו אדם הצביע לשני posts). שילפו באופן סימטרי כלומר a.b, b.a.

```
select v1.PostId, v2.PostId, count(*) as same_users_count
from votes as v1
join votes as v2
on v1.UserId = v2.UserId
where v1.PostId <> v2.PostId
group by v1.PostId, v2.PostId
having same_users_count > 10 ;
```

התחלנו מטבלת votes שמכילה את כל ההצבעות של המשתמשים. ביצענו self join של הטבלה לעצמה על פי מזהה המשתמש כדי למצוא פוסטים שעליהם אותו משתמש הצביע. לאחר מכן סיננו מקרים שבהם מדובר באותו פוסט - כי תמיד נקבל פוסט עם עצמו לכן לא יעזור לנו למצוא פוסטים דומים. קיבצנו לפי זוג הפוסטים וספרנו כמה משתמשים הצביעו לשניהם. לבסוף הצגנו רק זוגות פוסטים שעבורם יותר מעשרה משתמשים משותפים הצביעו לשני הפוסטים.